

# 处理器基本工作模型与指令级并行

Processor Execution Model and Instruction-Level Parallelism

主讲人：焦庆春、刘德康

自动化与电气工程学院

智能科学与技术系

**核心问题：一条指令究竟怎样通过时钟、寄存器和数据通路执行？如果提高频率受到限制，能否让已有硬件在同样时间内完成更多工作？**



## 上一节：系统约束

---

存储墙：数据能否及时送到

功耗墙：频率还能提高多少

ILP墙：有多少独立工作可做

## 本节：内核效率

---

缩短每个时钟周期的关键路径

重叠不同指令的执行阶段

减少资源冲突和依赖等待

- |    |                |                           |
|----|----------------|---------------------------|
| 01 | 时钟同步、指令周期与数据通路 | 讲义2.1.1、2.1.2 · P25 – P28 |
| 02 | 单周期与多周期处理器     | 讲义2.1.3 · P28 – P29       |
| 03 | 流水线与性能计算       | 讲义2.2.1 · P30 – P31       |
| 04 | 三类冒险及处理方法      | 讲义2.2.2 · P31 - 33        |

## 什么是时钟信号？

它是由处理器内部时钟发生器产生的周期性电信号，在高电平和低电平之间规律振荡，为所有操作提供统一的时间基准



### 系统同步

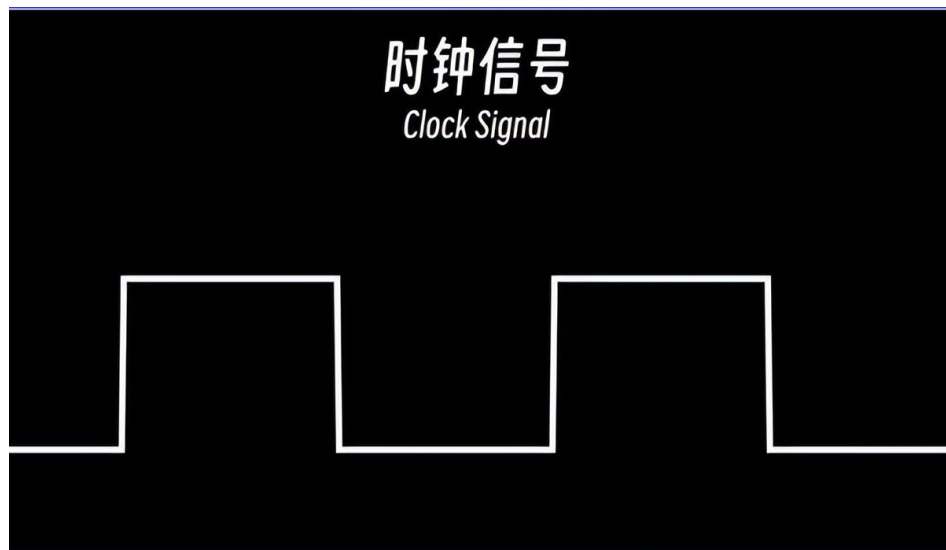
指挥处理器内部数以亿计的晶体管协同工作，保证数据传输与处理的时序正确性



### 基本节拍

定义了处理器执行每一步操作的最小时间单位，决定了指令执行的速度上限





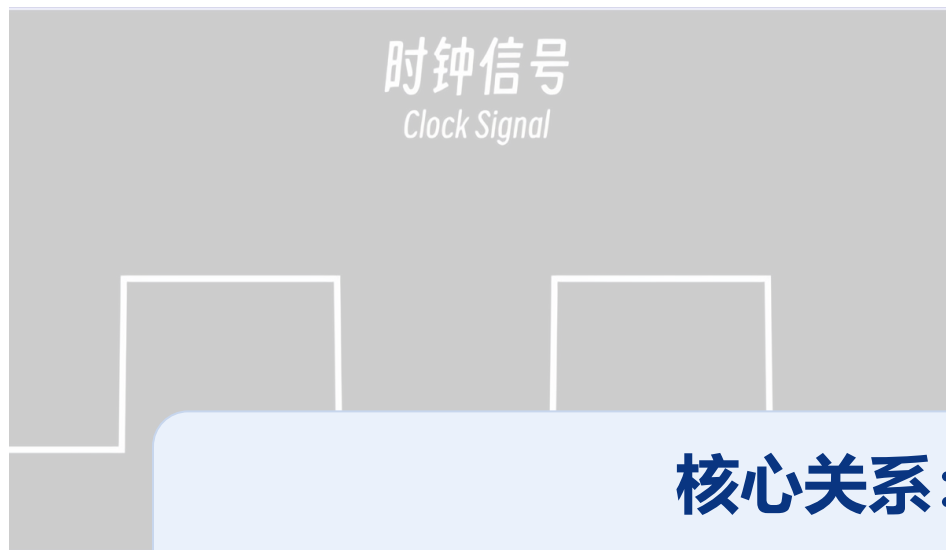
**标准的方波时钟信号示意图，处理器的所有同步操作均以此信号的上升沿或下降沿为基准，是数字电路的“心跳”**

## 时钟周期

定义为相邻两个时钟脉冲上升沿之间的时间间隔，是处理器执行最基本操作的最小时间单位。以纳秒 (ns) 为计量单位

## 主频

时钟信号每秒钟振荡的次数，常用单位为兆赫 (MHz) 或吉赫 (GHz)。主频直接反映了处理器的时钟速度，是衡量运算性能的重要指标



标准的方波时钟信号示意图，处理器的所有同步操作均以此信号的上升沿或下降沿为基准，是数字电路的“心跳”

**核心关系：主频 = 1 / 时钟周期**  
**(主频越高，时钟周期越短，单位时间内可完成的操作越多)**

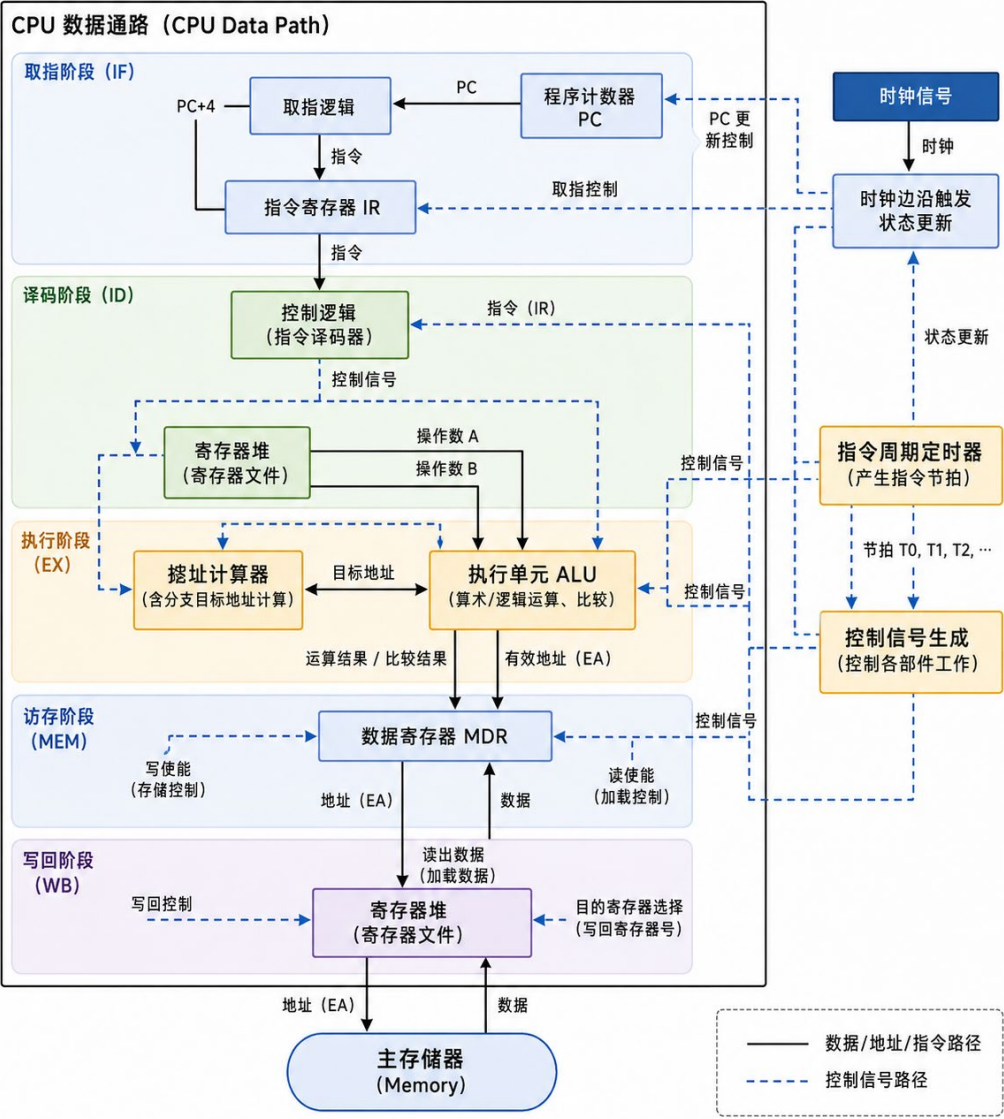
时钟周期

主频

定义为相邻两个时钟脉冲上升沿之间的时间间隔，是处理器执行最基本操作的最小时间单位。以纳秒 (ns) 为计量单位

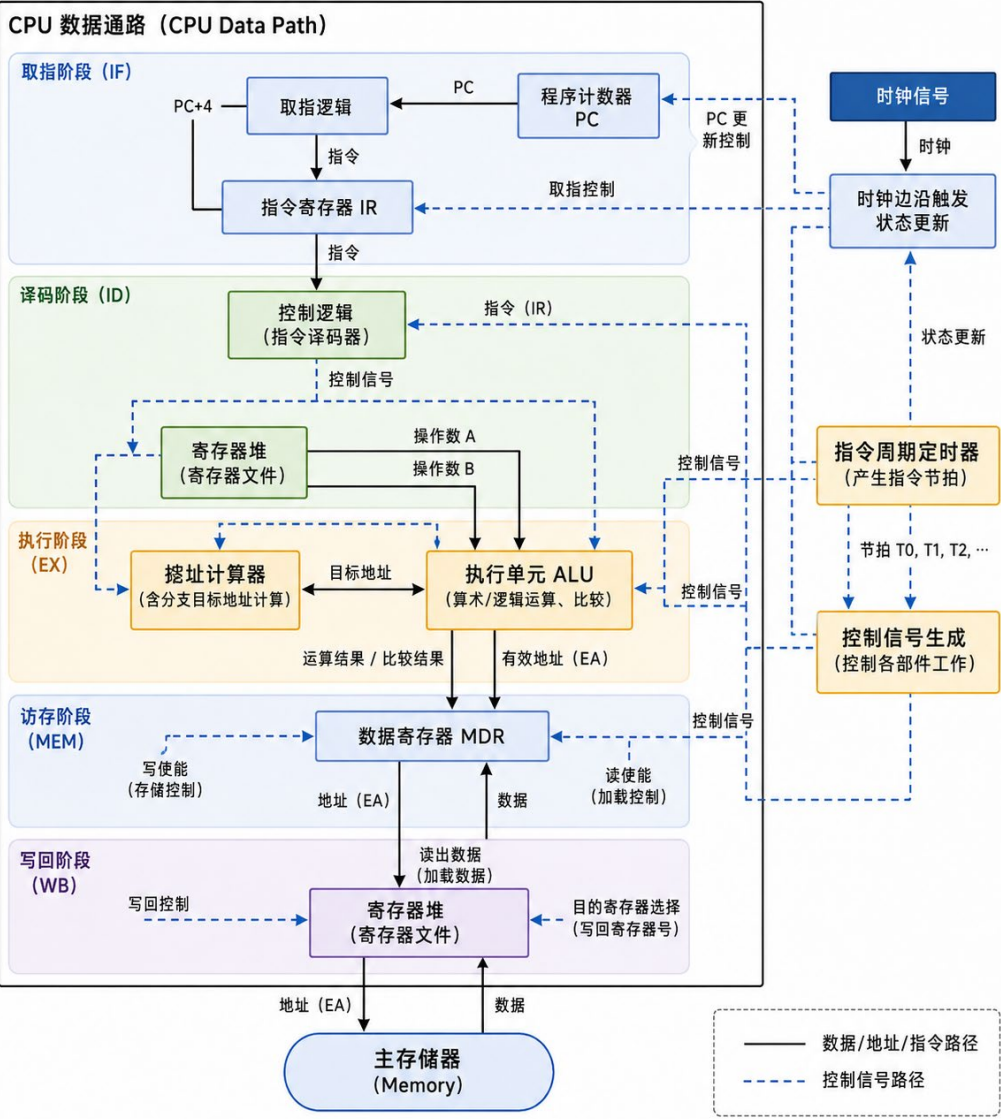
时钟信号每秒钟振荡的次数，常用单位为兆赫 (MHz) 或吉赫 (GHz)。主频直接反映了处理器的时钟速度，是衡量运算性能的重要指标





说明：图中各寄存器（PC、IR、MDR、寄存器堆等）在时钟边沿触发下更新状态。

- **处理器内部时钟发生器**：产生周期性的电信号，即时钟信号。每一次从低到高或从高到低的跳变，都标志着下一个时间单元的开始
- **作用**：同步整个芯片上不同部件的操作，确保某个部件输出数据时，接收该数据的部件已经准备好读取它
- **指令周期的基本阶段**：取指、译码、执行、访存、写回。几乎所有现代处理器，指令执行过程都可以抽象为五个基本阶段的循环



说明：图中各寄存器（PC、IR、MDR、寄存器堆等）在时钟边沿触发下更新状态。

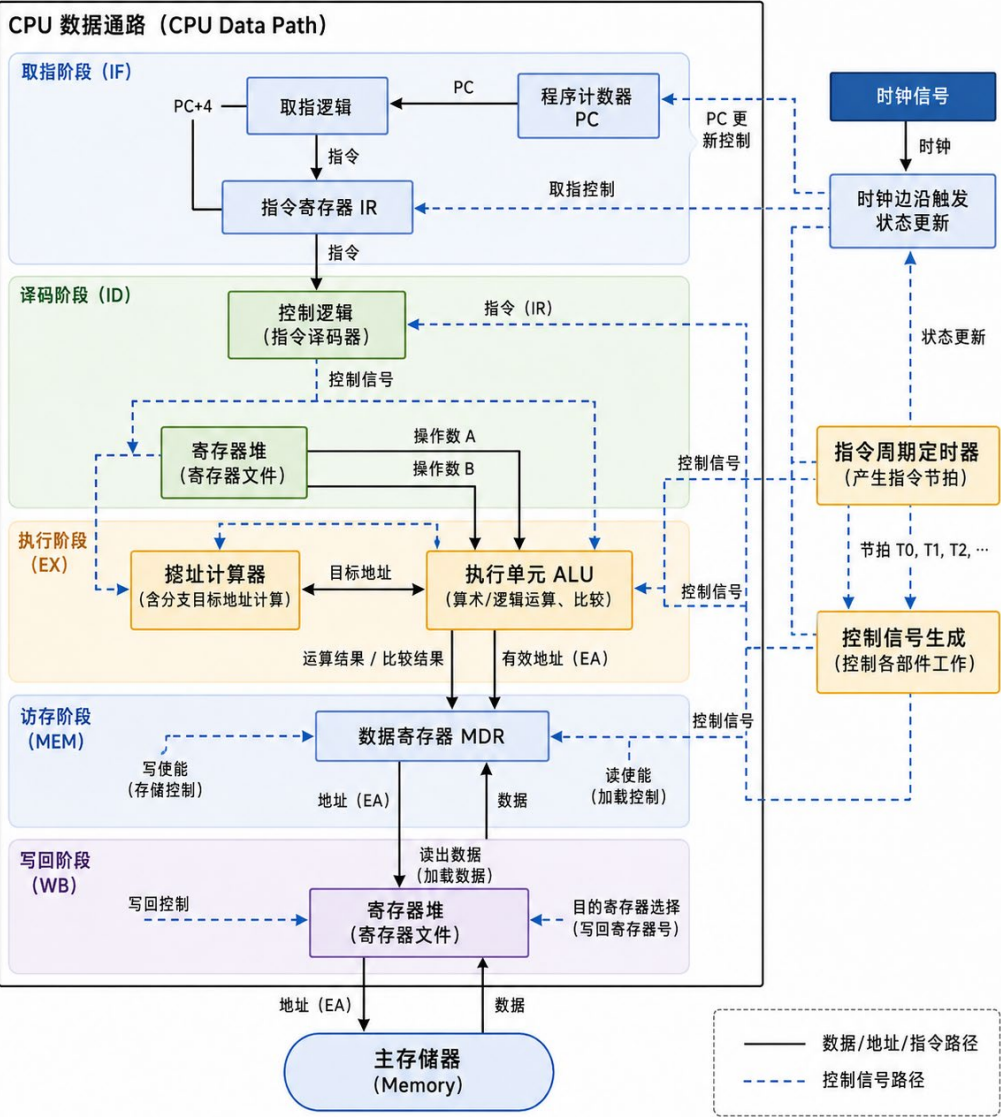
## 数据通路的核心组件

**寄存器堆：** 提供了一组被统一命名的存储位置，用于临时存放指令执行过程中最频繁访问的数据和地址。读写速度极快，一个指令周期内可完成访问

**算数逻辑单元：** 负责执行所有的算数运算（加减乘除）和逻辑运算（与或非、异或）。从寄存器堆或其它来源获取操作数，进行处理后将结果输出

**内存接口：** 连接处理器核心与主存储器的桥梁。加载指令/存储指令经此接口读入/写回





说明：图中各寄存器（PC、IR、MDR、寄存器堆等）在时钟边沿触发下更新状态。

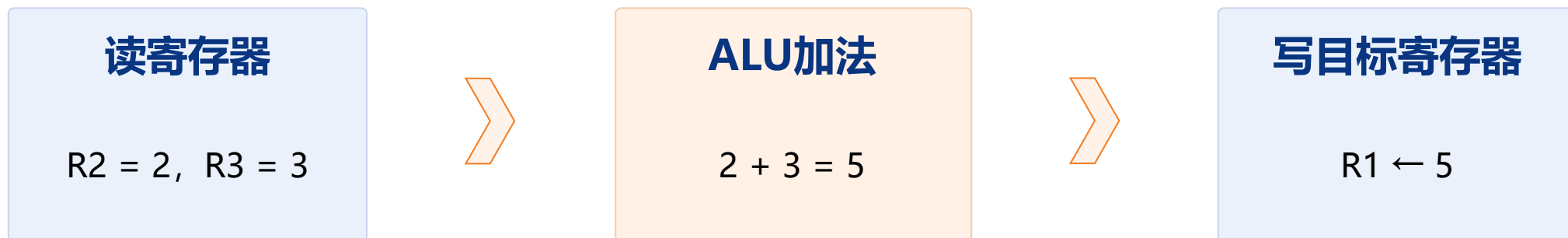
控制单元的功能

根据指令译码产生控制信号序列

- 在指令周期的译码阶段，控制单元会解析指令寄存器中的操作码和操作数，并将其作为输入，通过内部复杂的组合逻辑产生一整套控制信号
- 例如，控制信号决定ALU本次执行加法还是减法操作，会选通寄存器堆中具体的源寄存器和目标寄存器，控制多路选择器的数据流向，管理内存接口的读写使能
- 控制单元确保了数据通路能够按照指令的语义要求，在正确时机、地点执行正确操作

| 阶段     | 主要任务         | 典型信息      |
|--------|--------------|-----------|
| IF 取指  | 根据PC读取指令     | 指令编码、下一PC |
| ID 译码  | 识别操作并读取寄存器   | 操作码、源操作数  |
| EX 执行  | 运算或计算有效地址    | ALU结果、地址  |
| MEM 访存 | 按指令需要读写数据存储  | 加载值、存储数据  |
| WB 写回  | 按指令需要更新目标寄存器 | 运算结果或加载值  |

## ADD R1, R2, R3 (将寄存器R2、R3中的数据相加写入R1寄存器)

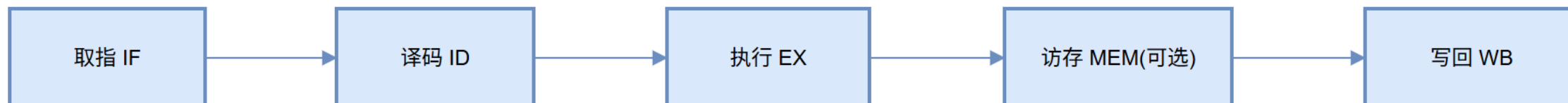


**取指、译码、执行、访存（可选）、写回**

### 同步数字系统的时序约束与稳定性

系统设计必须保证，在任何一级逻辑电路中，信号从输入到输出稳定所需的时间（即传播延迟），以及寄存器建立和保持时间的要求，都能在一个始终周期内得到满足

- **所有指令的执行都被限定在一个固定的时钟周期内完成**
- **时钟周期长度须足以让最复杂、最耗时的指令能够完成全部操作**
- **(从取值开始，经过译码、执行、访存到最终写回)**



### 控制逻辑极其简单

每条指令的执行路径都是确定且固定的

### “木桶效应” 严重制约

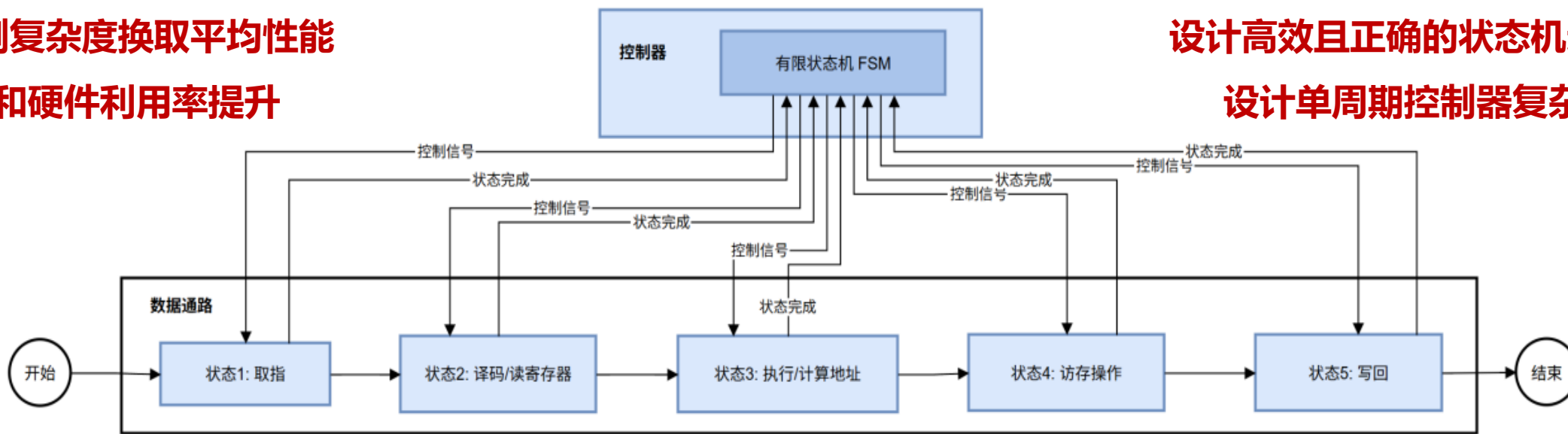
时钟周期为所有指令中的最慢的指令而设计，导致绝大多数简单指令在很早就完成了计算，却必须等待漫长的时钟周期结束

**以最低性能指令为标准设定系统步调，显著降低处理器整体平均性能**

- 多周期处理器将指令执行分解为多个可变长的时钟步骤
- 允许不同指令、甚至同一指令的不同执行阶段，占用不同数量的时钟周期
- 优势：资源共享与频率提升。支持部件复用，每个时钟周期逻辑操作减少，可以采用更高时钟频率

控制复杂度换取平均性能  
和硬件利用率提升

设计高效且正确的状态机远比  
设计单周期控制器复杂

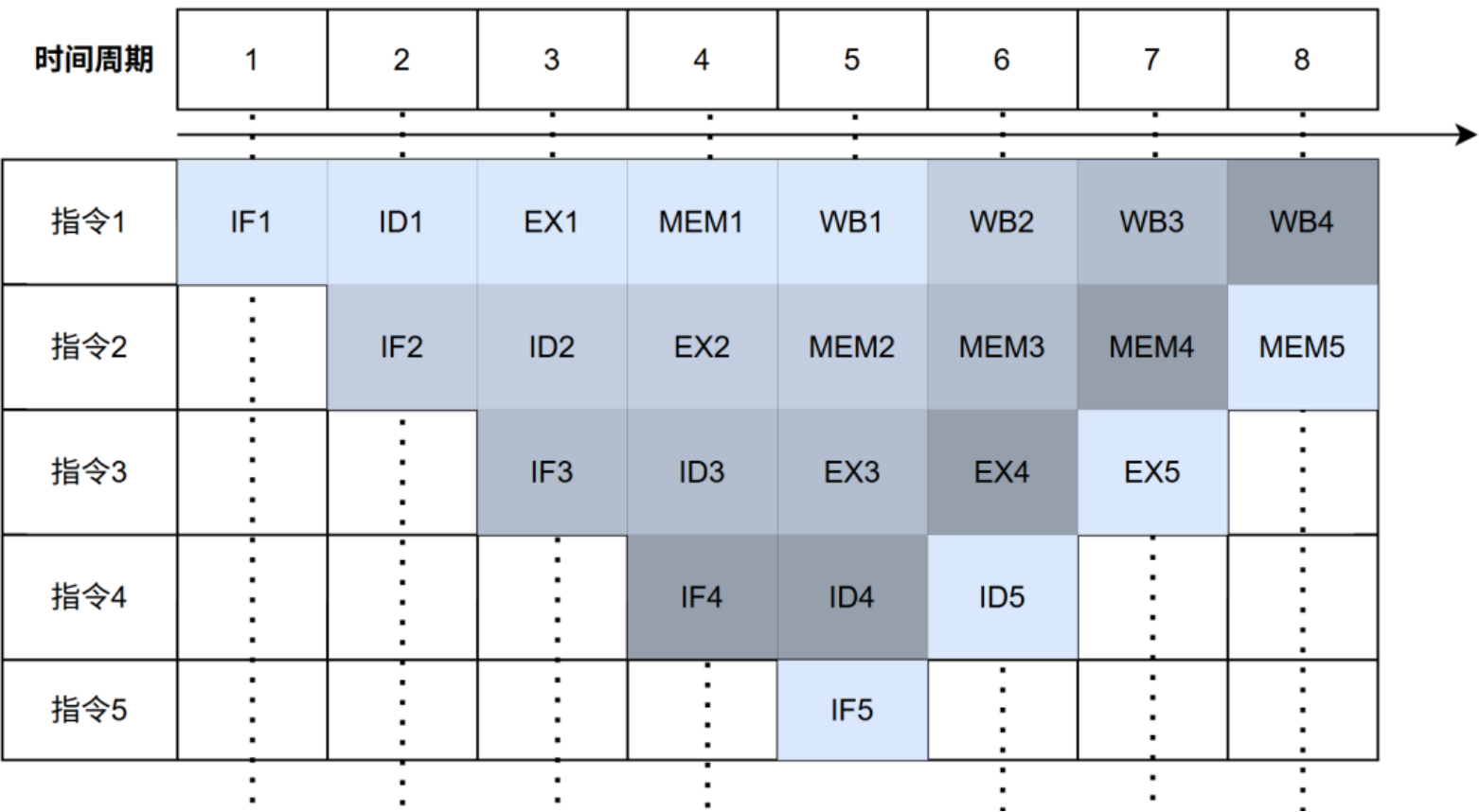


性能的再平衡：多周期引入额外开销与控制复杂度

每个时钟周期之间需要插入状态寄存器来暂存中间结果；控制逻辑复杂度急剧上升，有限状态机



□ 指令流水线技术：通过**让多条指令在时间上重叠**，显著提高处理器指令吞吐率



五级流水线指令重叠执行的时序示意图

**没有流水线的处理器：**一条指令必须完整地经历IF、ID、EX、MEM、WB五个阶段后，下一条指令才能够开始执行

**流水线技术：**将这个串行过程分解为多个独立的阶段，并让这些阶段在时间上并行工作

**时间上的重叠使得处理器在理想情况下每个时钟周期都能完成一条指令的执行**

**引入空间上的重复硬件来换取时间上的高效利用**

- **理想流水线**：假设所有指令在各个阶段的处理时间完全相等；  
假设流水线可以持续不断地充满，不存在任何中断或停顿
- **现实情况**：指令之间存在**数据依赖**（后续指令需要等待前一条指令的结果）和**控制依赖**（如分支指令改变程序流向）会频繁导致流水线产生“气泡”（**空闲周期**）
- **理想流水线**：流水线级数越多，每一级需要完成的逻辑操作越简单，因此可以设计更高的时钟频率
- **现实情况**：更深的流水线意味着分支预测错误或其它冒险事件发生时，需要**清空更多的在途指令**；每一级流水线寄存器本身都会引入**额外的时序开销和功耗**

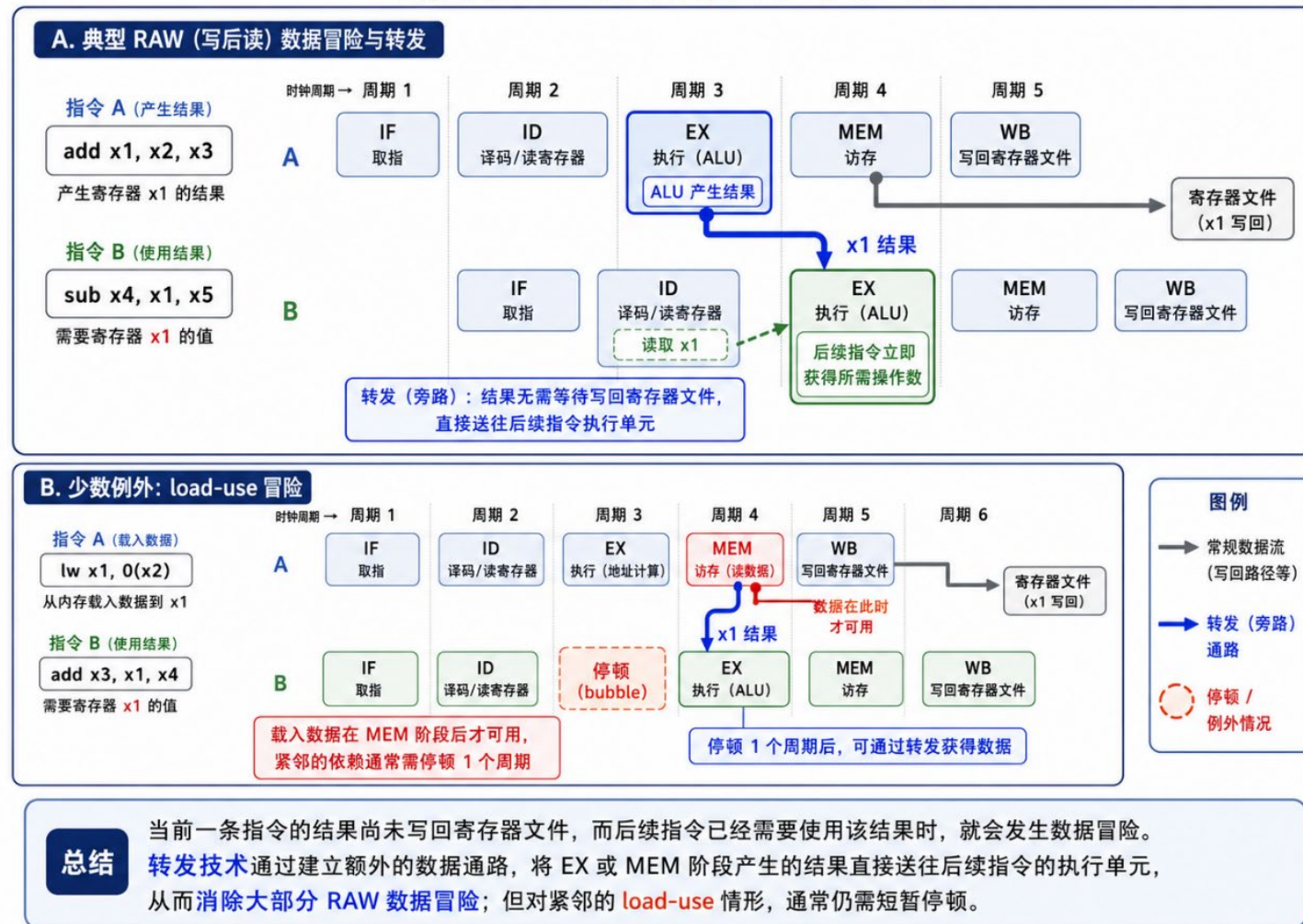
**实际流水线效率远低于理论值；加深流水线来提升频率的边际效益递减**

**数据冒险：**当一条指令需要读取某个寄存器的值，而该值正由前面尚未完成的指令进行写入时，便发生了数据冒险。最典型的情况是**写后读冒险**，即后续指令需要读取前一条指令的计算结果

### 解决方案：转发技术（旁路）

处理器内部建立额外的数据通路，将一条指令在执行阶段或访存阶段产生的结果，直接转发给需要改结果作为输入的后续指令的执行单元，无需等待写回寄存器

数据冒险与转发技术示意图



**控制冒险：**由改变程序执行流程的指令引起，其中最主要的是条件分支指令。

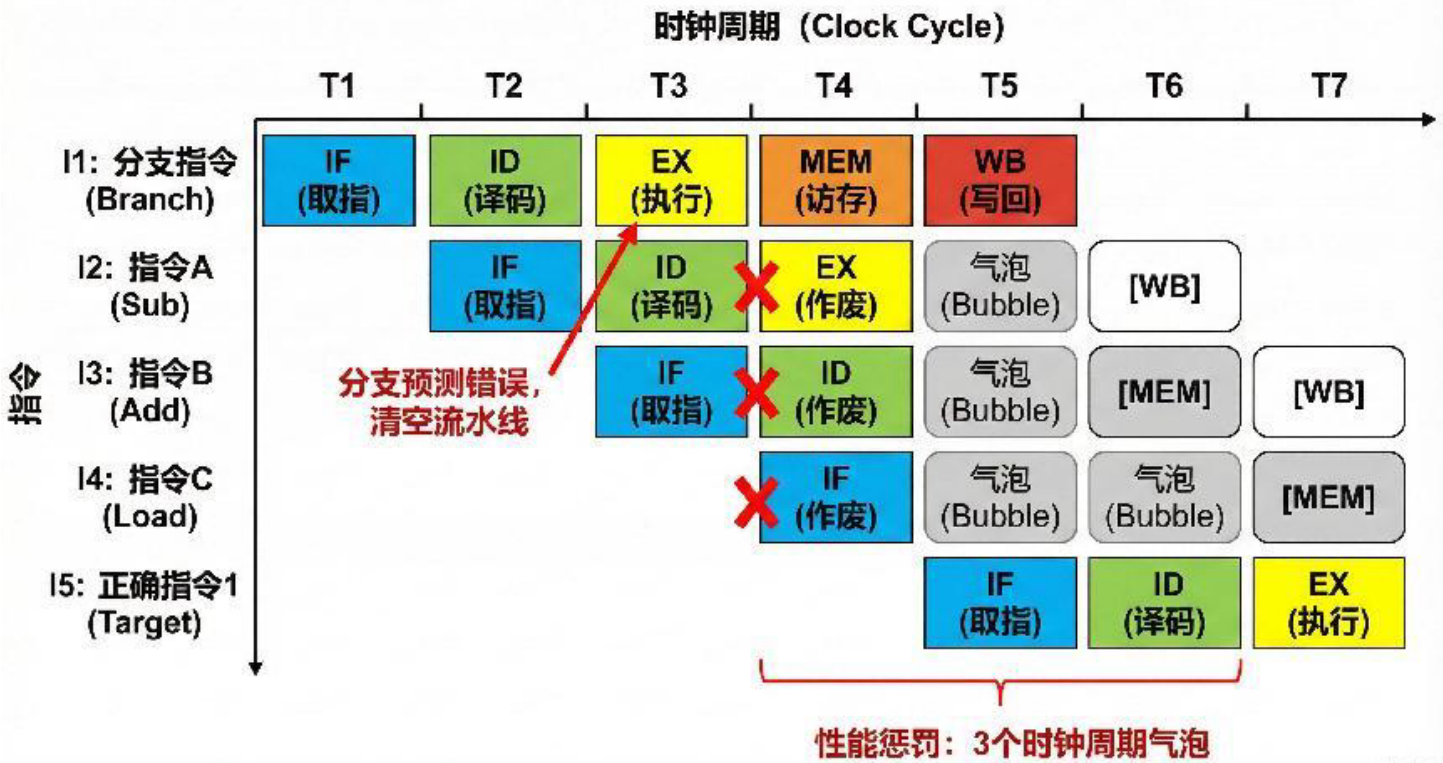
**解决方案：分支预测技术**

在分支指令的结果尚未明确之前，预测其执行方向（跳转或不跳转），并沿着预测的路径继续取指和执行指令

现代处理器分支预测准确率 > 95%

预测正确：流水线满负荷状态

预测错误：清空指令，正确地址重新开始

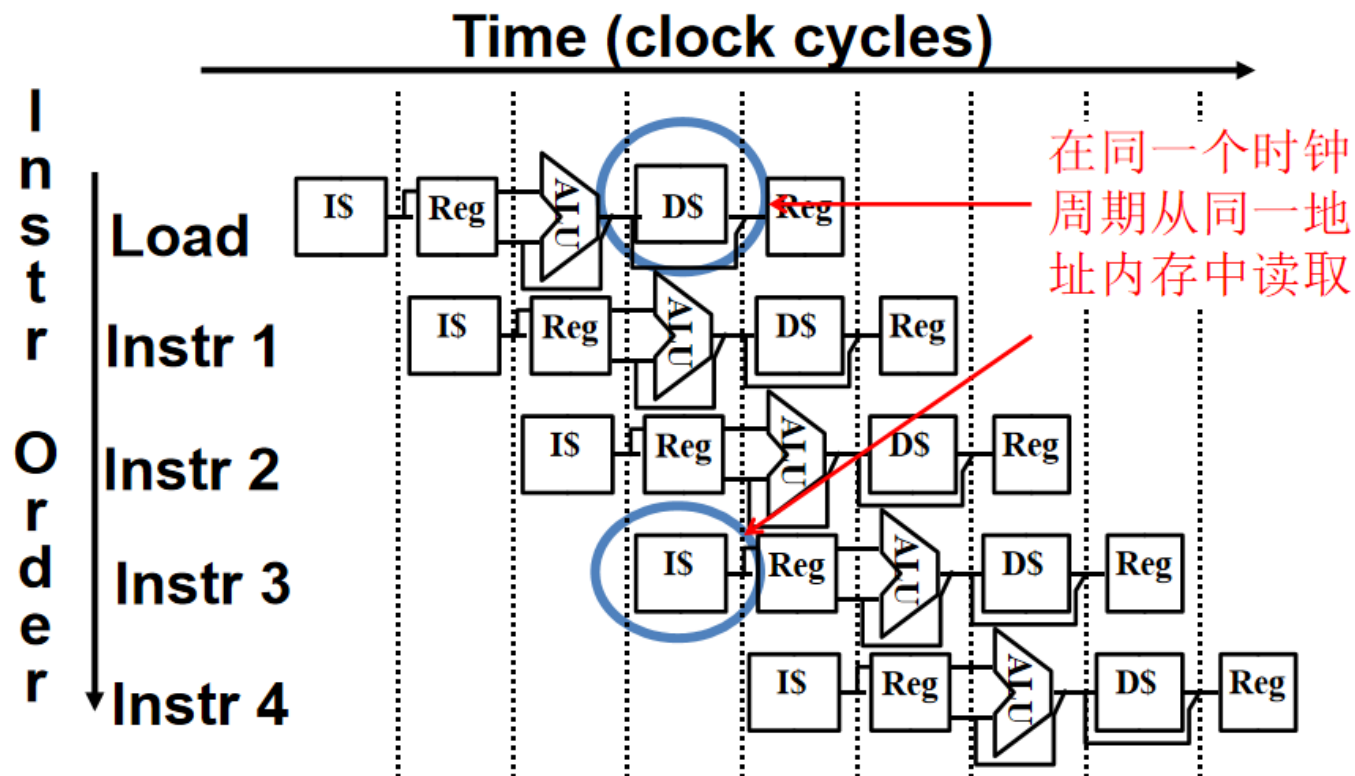




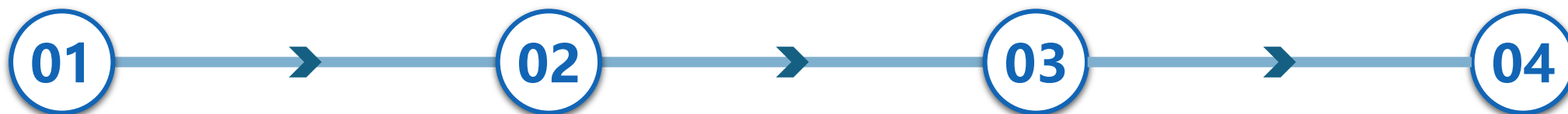
**结构冒险：**当流水线中的多个阶段需要访问同一个物理资源时，就会发生结构冒险

**解决方案：**资源重复的核心思想

采用分离的指令缓存和数据缓存，为取指和访存阶段提供独立的存储访问端口，从而消除对单一存储端口的竞争。为处理器执行阶段配备多个算术逻辑单元或地址计算单元，使得多条指令能够在同一周期内执行运算，解决执行部件资源冲突



**解决方案：**使用独立的 L1 I\$ 和 L1 D\$



**时钟与数据  
通路**

**单周期与多  
周期**

**流水线**

**三类冒险**



**规定状态何时更新、  
数据怎样处理**



**比较关键路径、资  
源复用**



**重叠不同指令，提  
高吞吐率**



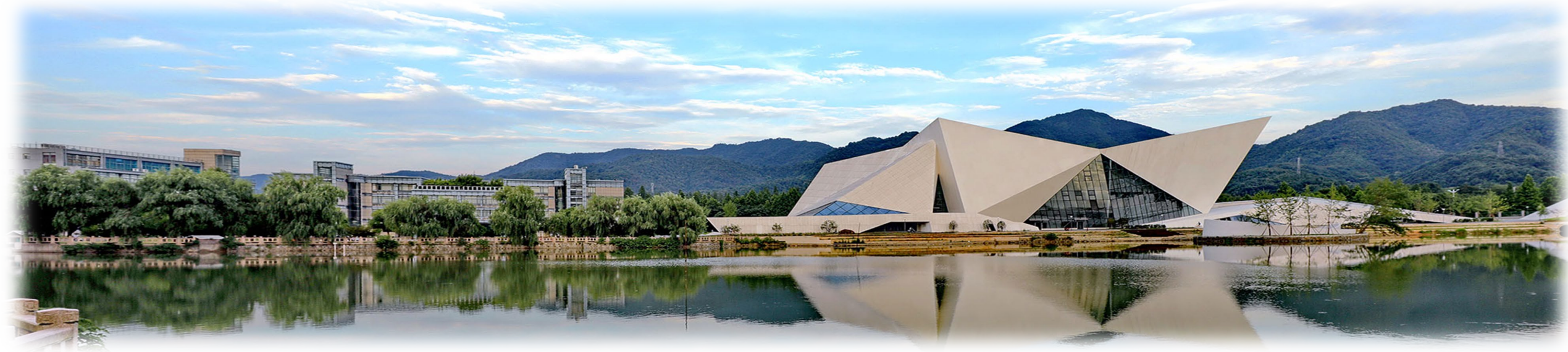
**按资源、数据和控  
制原因处理等待**

**思考** 五级流水线为什么不一定加速五倍？



谢谢

Thank you



# 从指令级并行（ILP）迈向RISC-V模块化架构

Instruction-Level Parallelism and Modular RISC-V

主讲人：焦庆春、刘德康

自动化与电气工程学院

智能科学与技术系

**核心问题：（1）如何进一步提高单线程吞吐率？**

**（2）处理器内部组织方式发生很大变化时，软件为什么仍可以保持正确执行？**



## □ 从“空间并行”到“动态调度”的演进

---

### 01 超标量架构

讲义2.3.1 · P34 – P36

### 02 乱序执行技术

讲义2.3.2 · P36 – P38

### 03 指令集架构的设计权衡

讲义2.4 · P38 – P43



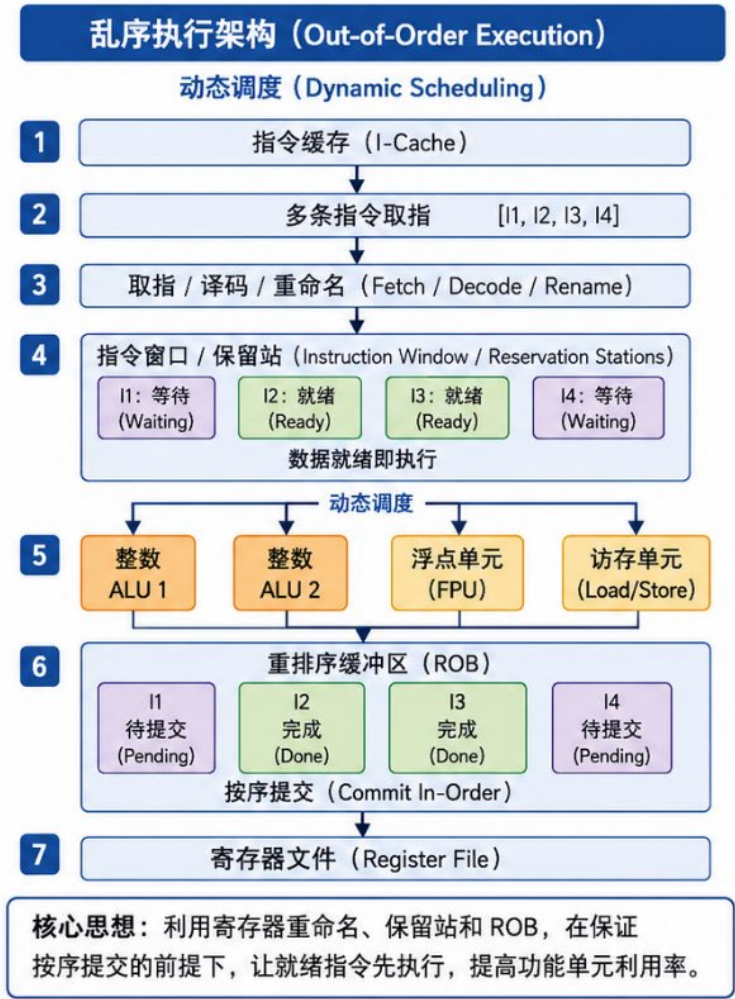
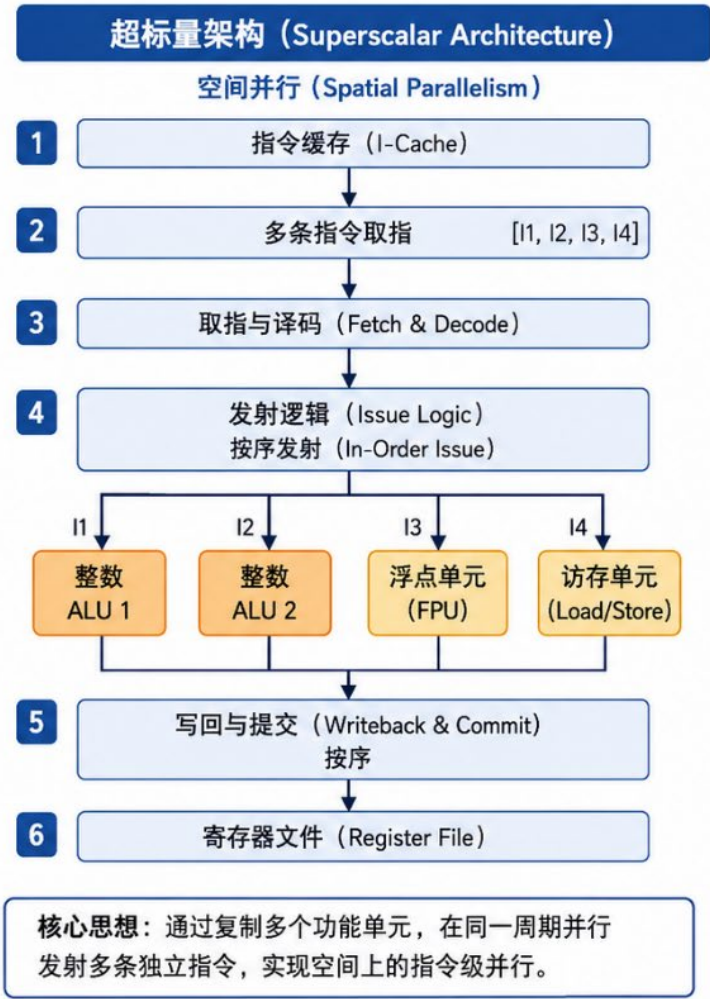
**超标量：**指 CPU 在一个时钟周期内获取、执行和提交多条指令

**标量：**指 CPU 在一个时钟周期内获取、执行和提交一条指令

**顺序：**顺序发射、顺序执行，指 CPU 按照指令原始顺序逐条发射、逐条执行

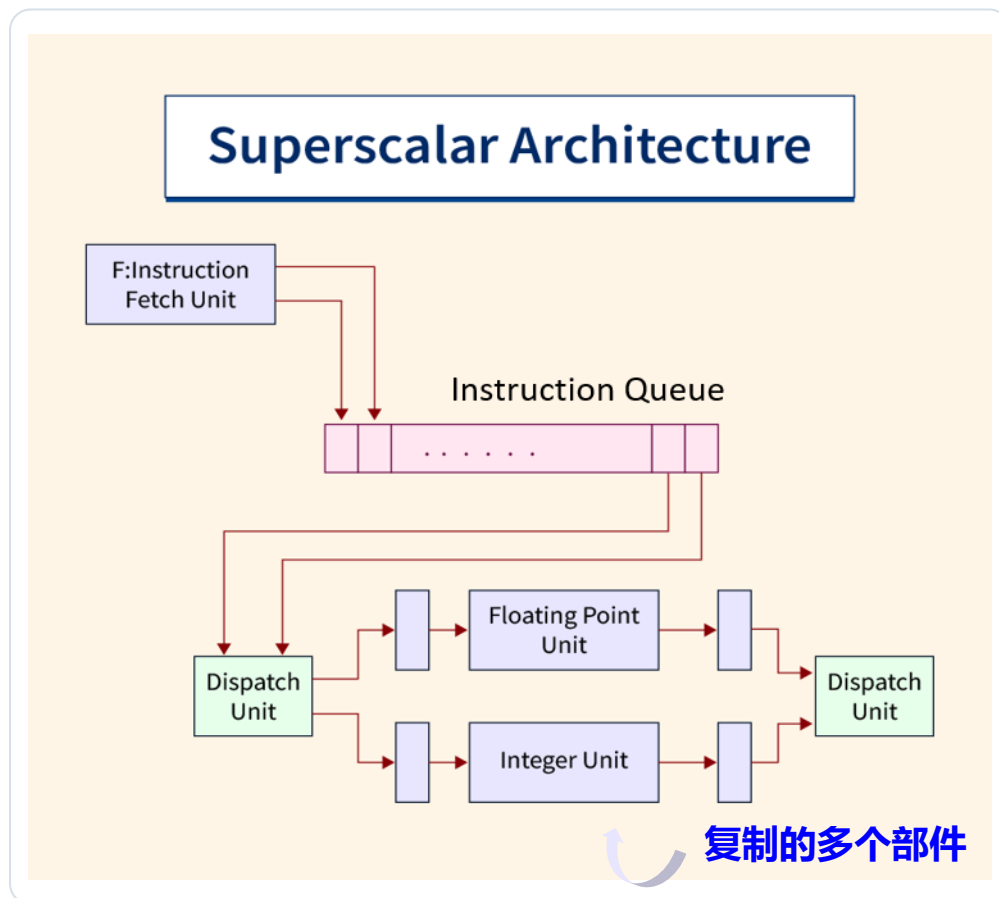
**乱序：**乱序发射、乱序执行

“超标量”一般和“乱序”搭配，“标量”一般和“顺序”搭配，前两者是现代高速微处理器所广泛使用的技术



设计哲学对比：左侧强调“增加并行硬件”，右侧强调“通过动态调度挖掘并行性”。

- 动态多发射技术，在每个时钟周期内，可以同时并发多条独立的指令
- 通过**复制核心执行部件并引入复杂的调度逻辑**，指令级并行的又一次飞跃



## 处理器设计从单纯的时序重叠迈入空间并行时代

超标量处理器的核心特征是能够在单个时钟周期内从指令缓存中取出多条指令，并将它们同时发射到多个独立的功能单元中执行：**多端口指令缓存；更宽的内部数据通路**

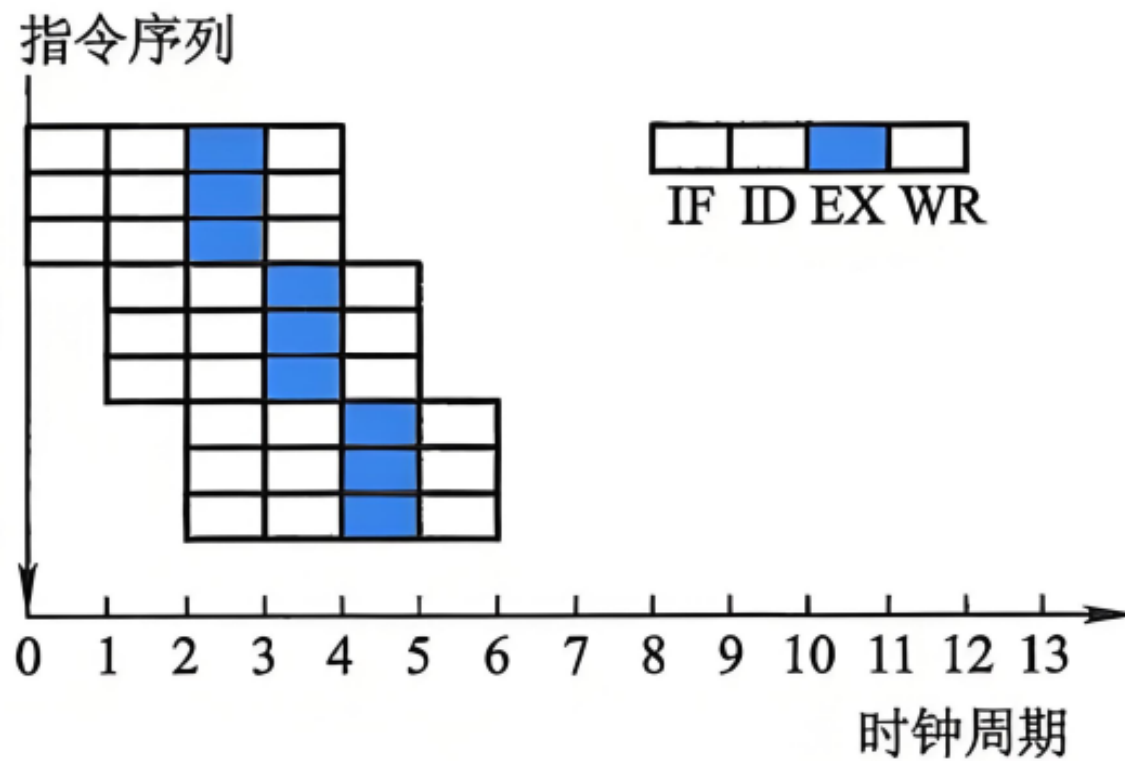
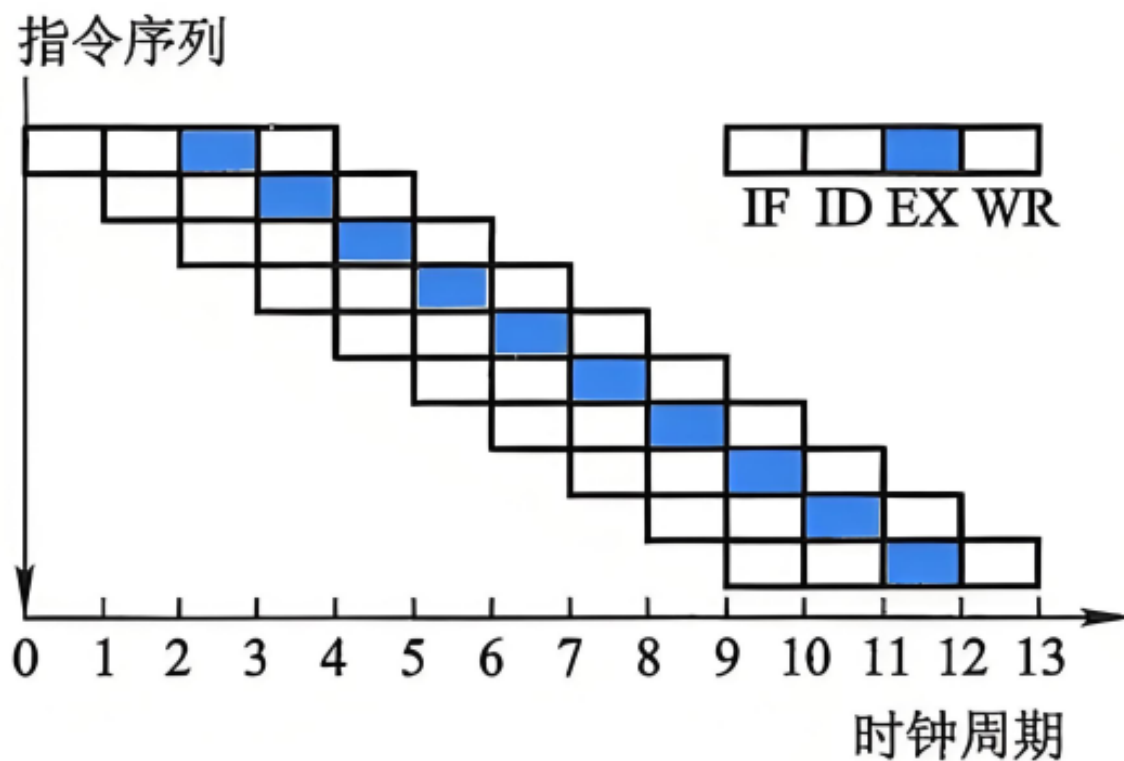
- 指令调度逻辑的设计复杂度：**指令调度器**

处理器必须动态地分析指令流中的数据依赖和控制依赖  
随着发射宽度从2路增加到4路、6路甚至更多，调度器需要检查的指令组合数量急剧增加，相关的比较逻辑和状态维护电路的复杂度呈指数级上升



□ 普通四级流水线（左）；超标量流水线（右）

□ （左）：每个时钟产生一条指令的结果；（右）每个时钟周期产生多条指令结果



- 突破指令间数据依赖对并行度的限制，是超标量架构实现高性能的关键
- 通过动态地调整指令的执行顺序，使得处理器最大限度地利用可用的功能单元



按序执行



乱序执行



按序执行



乱序执行

LDR R1, [R0];  
ADD R2, R1, R1;  
ADD R4, R3, R3;

处理器取出一批指令，分析找出  
哪些不存在相互依赖的指令，同  
时将其发射到多个逻辑单元执行

第二条指令和第一条存在依赖关系  
第二条指令和第三条指令无关  
第一、第二条按序；第三条与前两条并行

- 寄存器重命名通过将逻辑寄存器动态映射到物理寄存器，**消除假数据依赖**
- 消除指令间**写后写**、**读后写**这两种假数据冒险；**真冒险的指令必须顺序执行**

### RAW 写后读

```
ADD R3, R1, R2
AND R5, R3, R4
```

### True Dependency

### WAW 写后写

```
ADD R3, R1, R2
ADD R5, R3, R4
AND R3, R6, R7
ADD R9, R3, R8
```

### Rename: R3→R10

```
ADD R3, R1, R2
ADD R5, R3, R4
AND R10, R6, R7
ADD R9, R10, R8
```

### WAR 读后写

```
ADD R1, R3, R2
ADD R3, R5, R4
```

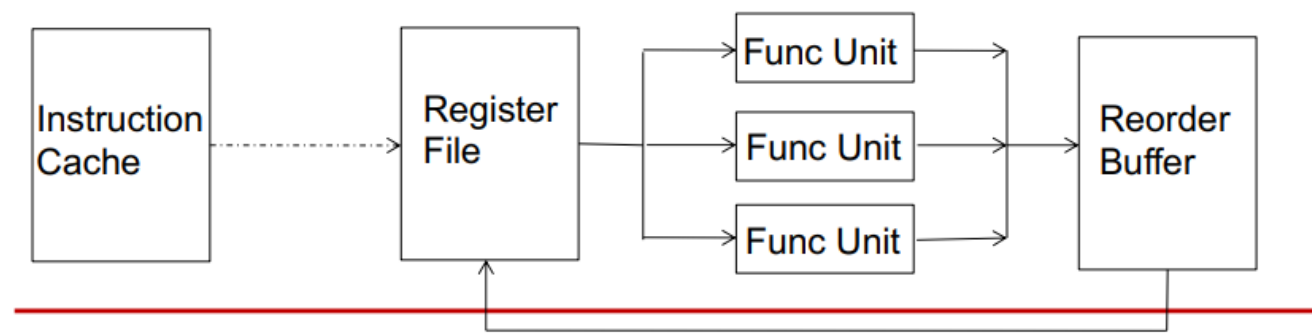
### Rename: R3→R10

```
ADD R1, R3, R2
ADD R10, R5, R4
```

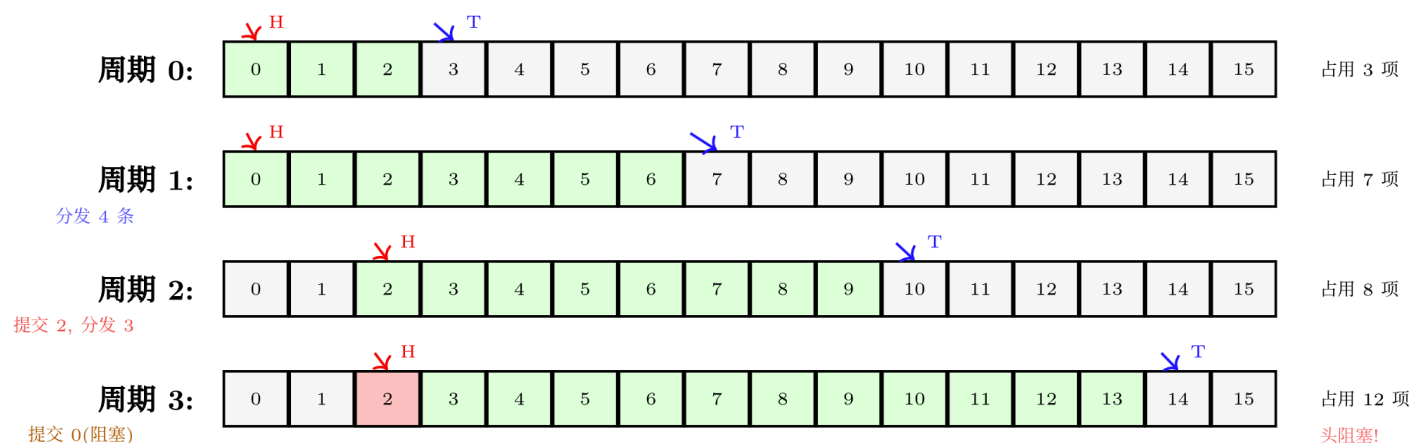
### 修改目的寄存器的名字

在重命名的过程中，处理器首先根据指令的源寄存器编号查询“**重命名映射表**”。重命名映射表记录了逻辑寄存器和物理寄存器的对应关系（物理寄存器是一些逻辑寄存器之外的、指令集规定之外的寄存器，设计来实现“无限大”的寄存器堆），重命名映射表就记录了这个映射关系。通过映射结果，指令可以找到源寄存器对应的物理寄存器，从而得到需要的数据。接下来，指令从空闲寄存器列表中找到空闲的物理寄存器，用来和指令的目的寄存器建立映射关系。从而消除虚假依赖，允许乱序执行

- 虽然指令可以乱序执行，但是**执行结果必须按照原始程序顺序提交**
- 重排序缓冲区是实现这一机制的核心部件，**先进先出的缓冲区（循环队列）**



重排序缓冲区核心思想：记录下指令在程序中的顺序，一条指令在执行完毕之后不会立刻提交（这里的提交指的是修改处理器状态，如修改逻辑寄存器堆），而是先在Buffer中等待，等到前面的所有指令都提交完毕，才可以提交结果到逻辑寄存器堆

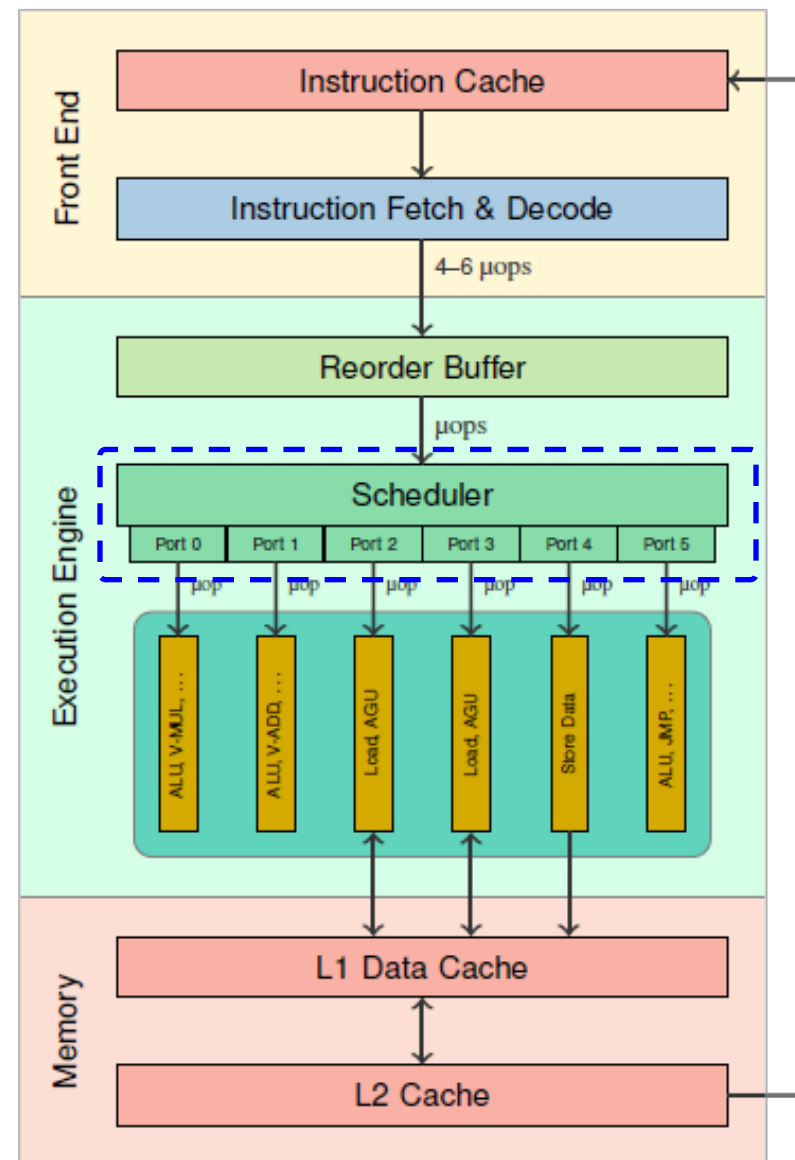
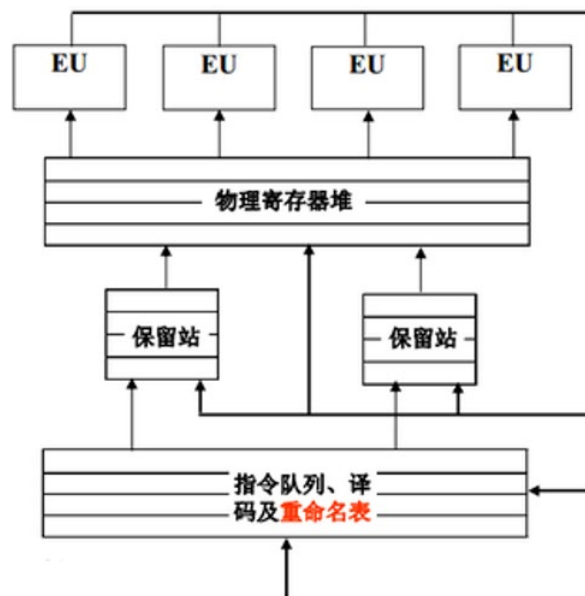




□ 保留站与调度器的设计构成了乱序执行引擎的**调度中枢**，负责实现指令的动态调度

□ 保留站位于功能单元之前，是**容纳已就绪或等待操作数的指令的缓冲区**

- 当指令经过重命名后，会被分发到对应功能单元的保留站中
- 保留站持续监听其内部指令的所有操作数是否已经就绪
- 一旦某条指令的所有操作数都可用，该指令被标记为“就绪”状态
- 调度器不断扫描所有保留站，将就绪的指令，在对应的功能单元空闲时将其发射执行



### 分支预测：先按预测前进，再用真实结果验证

分工：分支预测选择路径；乱序执行优先安排已就绪的指令。

#### ① 预测正确：减少等待

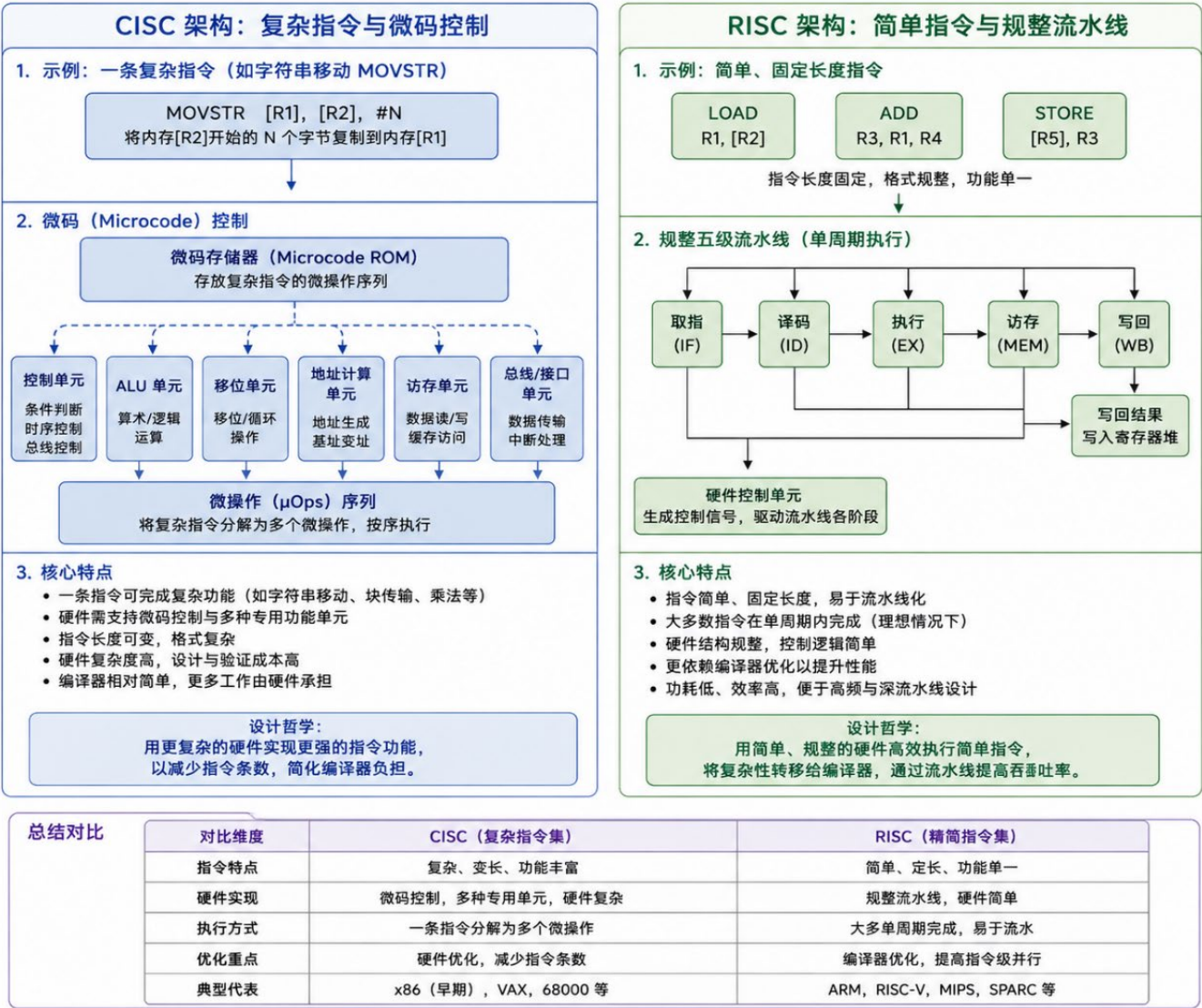


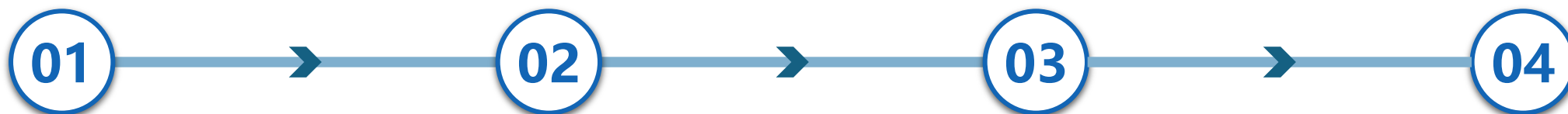
#### ② 预测错误：丢弃错误路径，重新开始



A、B 表示两条分支路径；恢复过程为示意，不对应固定的周期数。

- ❑ 指令集是CPU中用于计算和控制计算机系统的一组指令集合。它定义了CPU支持的指令格式、操作功能以及操作数的获取方式。指令集是软件与硬件之间的桥梁，决定了CPU的功能和性能
- ❑ 指令集由操作码和操作数组成。操作码定义指令具体功能，例如加法、移动数据等；操作数指定操作对象，可以是寄存器、内存地址或立即数。例如，汇编指令 MOV AX, 10 的操作码是 MOV，操作数是AX和10
- ❑ 指令集主要分为两类：**复杂指令集（CISC）**和**精简指令集（RISC）**
- ❑ **CISC**：指令数量多，支持复杂操作，指令长度可变，适合复杂任务。典型架构为x86
- ❑ **RISC**：指令数量少，专注于简单指令，指令长度固定，适合高性能计算。典型架构ARM





超标量架构

寄存器重  
命名

重排序缓  
冲区

指令集

扩大同周期可处理  
的指令宽度

保留真实依赖，允  
许乱序执行

保证乱序执行结果  
正确性

在明确接口上选择  
能力与实现

**思考** 重命名为什么不能消除RAW?



谢谢

Thank you

